Principles of Machine Learning

Prof. Christian Bauckhage



lecture 03

gradient-based optimization, gradient-free optimization, and gradient flows

machine learning is all about model fitting and model fitting is all about parameter optimization

often, there a no closed form solutions for optimal parameters so that we need to work with iterative estimation procedures

most prominent in machine learning are gradient-based optimization schemes so that we might as well study them early on . . .

in this lecture, we thus look at different flavors of gradient-based optimization . . . for the fun of it, we also mention the existence of gradient-free optimizers as well as the (foreshadowing) notion of gradient flows

outline

setting the stage

iterative least squares

gradient-based optimization

gradient-free optimization

(least squares) gradient flows

summary

setting the stage

given (labeled) *data*

$$D = \left\{ \left(\boldsymbol{x}_j, \boldsymbol{y}_j \right) \right\}_{j=1}^{n} \subset \mathbb{R}^m \times \mathbb{R}^t$$

and assuming some parametrized *model family*

$$f_\theta : \mathbb{R}^m \to \mathbb{R}^t$$

we may consider some *loss* $\mathcal{L}\left(\theta \,\middle|\, f_\theta, D\right)$ to estimate

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}}\ \mathcal{L}\left(\theta \,\middle|\, f_\theta, D\right)$$

this kind of *training* adjusts or fits the model to the data

last time, we assumed *linear models*

$$f_{\boldsymbol{w}}(\boldsymbol{x}) = \boldsymbol{\varphi}_{\boldsymbol{x}}^{\mathsf{T}} \boldsymbol{w}$$

together with the *least squares loss*

$$\mathcal{L}(\boldsymbol{w} \mid f_{\boldsymbol{w}}, D) = \sum_{j} \Big(f_{\boldsymbol{w}}(\boldsymbol{x}_j) - y_j \Big)^2$$

we saw that we may simply write the loss as

$$\mathcal{L}(\boldsymbol{w}) = \big\| \boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{w} - \boldsymbol{y} \big\|^2$$

and that the *unique minimizer* of this loss is

$$\hat{\boldsymbol{w}} = \operatorname*{argmin}_{\boldsymbol{w}} \big\| \boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{w} - \boldsymbol{y} \big\|^2 = \big[\boldsymbol{\Phi} \boldsymbol{\Phi}^{\mathsf{T}} \big]^{-1} \boldsymbol{\Phi}\, \boldsymbol{y}$$

today, we ignore models and losses but simplify things (and their notation)

to begin with, we shall consider quadratic functions $f : \mathbb{R}^n \to \mathbb{R}$ of the form

$$f(\boldsymbol{x}) = \left\|\boldsymbol{M}\boldsymbol{x} - \boldsymbol{v}\right\|^2 = \left[\boldsymbol{M}\boldsymbol{x} - \boldsymbol{v}\right]^\intercal \left[\boldsymbol{M}\boldsymbol{x} - \boldsymbol{v}\right] = \boldsymbol{x}^\intercal \boldsymbol{M}^\intercal \boldsymbol{M}\boldsymbol{x} - 2\,\boldsymbol{x}^\intercal \boldsymbol{M}^\intercal \boldsymbol{v} + \boldsymbol{v}^\intercal \boldsymbol{v}$$

given functions like these, we will ask for their minimizer which we write as

$$\boldsymbol{x}_* = \operatorname*{argmin}_{\boldsymbol{x}} f(\boldsymbol{x})$$

given what we already know about quadratic forms, we immediately realize

$$\boldsymbol{x}_* = \left[\boldsymbol{M}^\intercal \boldsymbol{M}\right]^{-1} \boldsymbol{M}^\intercal \boldsymbol{v}$$

however . . .



note

the solution $\boldsymbol{x}_* = \left[\boldsymbol{M}^\intercal \boldsymbol{M}\right]^{-1} \boldsymbol{M}^\intercal \boldsymbol{v}$ does *not* necessarily mean

$$\boldsymbol{M}\boldsymbol{x}_* = \boldsymbol{v}$$

it really only means that $\boldsymbol{x}_*$ is the best possible answer to

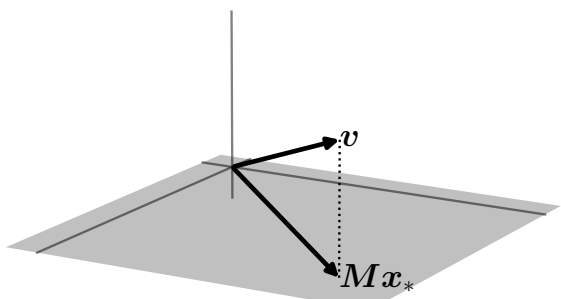
$$\boldsymbol{x}_* = \underset{\boldsymbol{x}}{\operatorname{argmin}} \left\|\boldsymbol{M}\boldsymbol{x} - \boldsymbol{v}\right\|^2$$

in general, we have parameters $\boldsymbol{M} \in \mathbb{R}^{m \times n}$ and $\boldsymbol{v} \in \mathbb{R}^m$ and solution $\boldsymbol{x}_* \in \mathbb{R}^n$

in general, $\boldsymbol{M}\boldsymbol{x}_*$ is the *orthogonal projection* of vector $\boldsymbol{v}$ into the space spanned by the columns of matrix $\boldsymbol{M}$

here is an example / illustration where

$$\boldsymbol{M} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} \quad \text{and} \quad \boldsymbol{v} = \begin{bmatrix} 4 \\ 4 \\ 2 \end{bmatrix}$$



for the special case of an invertible matrix $\boldsymbol{M} \in \mathbb{R}^{n \times n}$ and a vector $\boldsymbol{v} \in \mathbb{R}^{n}$, we have

$$\boldsymbol{x}_* = \boldsymbol{M}^{-1}\boldsymbol{v}$$

such that

$$\boldsymbol{M}\boldsymbol{x}_* = \boldsymbol{M}\boldsymbol{M}^{-1}\boldsymbol{v} = \boldsymbol{v}$$

moreover, in this (very) special case, the general LSQ solution is still valid because

$$\boldsymbol{x}_* = \left[\boldsymbol{M}^{\intercal}\boldsymbol{M}\right]^{-1}\boldsymbol{M}^{\intercal}\boldsymbol{v} = \boldsymbol{M}^{-1}\boldsymbol{M}^{-\intercal}\boldsymbol{M}^{\intercal}\boldsymbol{v} = \boldsymbol{M}^{-1}\boldsymbol{v}$$

note: this identity really only holds if $\boldsymbol{M}$ is square and invertible
but, for simplicity, the next slides focus on this setting ...



note

if n is large, computing $\boldsymbol{M}^{-1}$ is quite expensive $(\Leftrightarrow$ efforts between $O\left(n^{2.3}\right)$ and $O\left(n^{3}\right))$

if $\boldsymbol{M}$ is *ill-conditioned* ($\Leftrightarrow$ almost *singular*), computing $\boldsymbol{M}^{-1}$ is prone to numerical errors

if entries of $\boldsymbol{M}$ are very small or large, computing $\boldsymbol{M}^{-1}$ suffers from floating point limits

$\Rightarrow$ **for production level code, it is a very good idea to try to avoid matrix inversion**

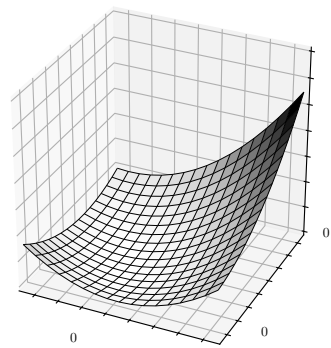
next, we will therefore look at *iterative least squares solving without matrix inversion*

for all that follows, it will be beneficial to visualize objective functions which is sadly only possible for simple or didactic settings where $\boldsymbol{x} \in \mathbb{R}^2$

for instance, for a didactic setting where $f : \mathbb{R}^2 \to \mathbb{R}$ with

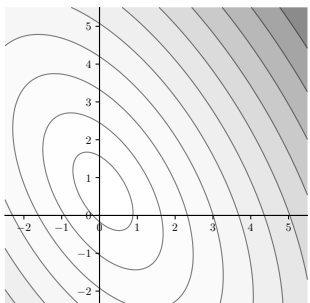
$$\boldsymbol{M} = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix} \quad \text{and} \quad \boldsymbol{v} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

we can visualize $f(\boldsymbol{x}) = \|\boldsymbol{M}\boldsymbol{x} - \boldsymbol{v}\|^2$ via a 3D surface plot

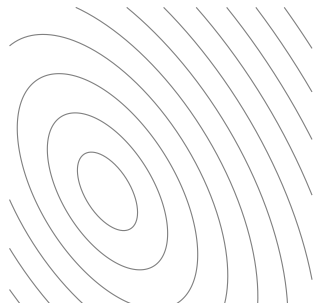


however, from now on, we will (always) visualize functions $f : \mathbb{R}^2 \to \mathbb{R}$ in terms of [contour | level set | isoline] plots

for example, for the above $f(\boldsymbol{x}) = \|\boldsymbol{M}\boldsymbol{x} - \boldsymbol{v}\|^2$, these may look like this



$\Leftrightarrow$



one more thing . . .

for every continuously differentiable scalar function $f : \mathbb{R}^m \to \mathbb{R}$, there exists an associated *vector field*, namely the gradient field

$$\boldsymbol{g} : \mathbb{R}^m \to \mathbb{R}^m$$

where

$$\boldsymbol{g}(\boldsymbol{x}) = \nabla f(\boldsymbol{x}) = \begin{bmatrix} \frac{\partial}{\partial x_1} f(\boldsymbol{x}) \\ \frac{\partial}{\partial x_2} f(\boldsymbol{x}) \\ \vdots \\ \frac{\partial}{\partial x_m} f(\boldsymbol{x}) \end{bmatrix}$$

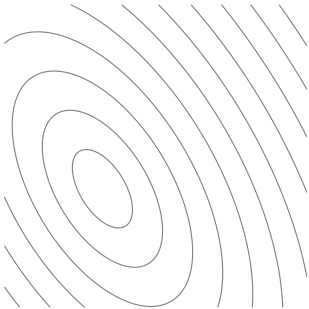
the vector $\boldsymbol{g}(\boldsymbol{x})$ gives the direction of steepest increase of f at $\boldsymbol{x}$

its magnitude $\|\boldsymbol{g}(\boldsymbol{x})\|$ indicates the corresponding rate of change

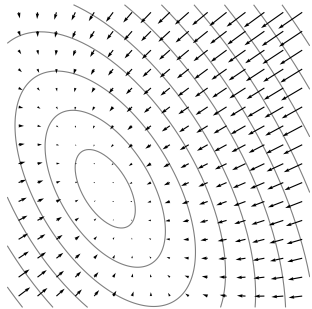
note

when *minimizing* a function $f : \mathbb{R}^m \to \mathbb{R}$, we are not interested in directions of steepest increase but of steepest decrease

this information is contained in the negative gradient $-\nabla f(\boldsymbol{x})$



function f



negative gradient field $-\nabla f$

iterative least squares

running example

in the following, we once again let

$$\boldsymbol{M} = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix} \quad \text{and} \quad \boldsymbol{v} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

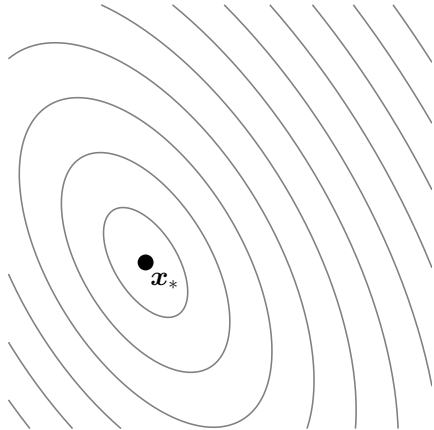
and consider the quadratic function

$$f(\boldsymbol{x}) = \left\| \boldsymbol{M}\boldsymbol{x} - \boldsymbol{v} \right\|^2$$

whose unique minimizer is given by

$$\boldsymbol{x}_* = \operatorname*{argmin}_{\boldsymbol{x}} \left\| \boldsymbol{M}\boldsymbol{x} - \boldsymbol{v} \right\|^2 = \boldsymbol{M}^{-1}\boldsymbol{v}$$

$$\approx \begin{bmatrix} 0.09 \\ 0.64 \end{bmatrix}$$



we have

$$\boldsymbol{x}_* = \underset{\boldsymbol{x}}{\text{argmin}} \left\| \boldsymbol{M}\boldsymbol{x} - \boldsymbol{v} \right\|^2 = \underset{\boldsymbol{x}}{\text{argmin}}\ \boldsymbol{x}^\intercal \boldsymbol{M}^\intercal \boldsymbol{M}\boldsymbol{x} - 2\boldsymbol{x}^\intercal \boldsymbol{M}^\intercal \boldsymbol{v} + cnst$$

$$\Leftrightarrow \quad \boldsymbol{x}_* = \underset{\boldsymbol{x}}{\text{argmin}}\ \tfrac{1}{2}\boldsymbol{x}^\intercal \tilde{\boldsymbol{H}}\boldsymbol{x} - \boldsymbol{x}^\intercal \boldsymbol{h}$$

where matrix $\tilde{\boldsymbol{H}} = \boldsymbol{M}^\intercal \boldsymbol{M}$ is *square*, *symmetric*, and *positive definite*

“one can show” (do it yourself !!!) that $\boldsymbol{x}_*$ is a solution if and only if

$$\tilde{\boldsymbol{H}}\boldsymbol{x}_* = \boldsymbol{h}$$

claim (w/o proof)

the problem $\tilde{\boldsymbol{H}}\,\boldsymbol{x} = \boldsymbol{h}$ can be solved without explicit matrix inversion

this is because, given some initial $\boldsymbol{x}_0 \in \mathbb{R}^n$ and an appropriate *step size* $\eta_k \neq 0$, the sequence

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \eta_k \left[\boldsymbol{h} - \tilde{\boldsymbol{H}}\,\boldsymbol{x}_k \right] \tag{1}$$

converges to the the solution of $\tilde{\boldsymbol{H}}\,\boldsymbol{x} = \boldsymbol{h}$

sanity check

if (1) converges in finitely many steps,
then there exists some $k \in \mathbb{N}$ such that

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k$$

and therefore

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_{k+1} + \eta_k \left[\boldsymbol{h} - \tilde{\boldsymbol{H}} \boldsymbol{x}_{k+1} \right]$$

$$\Leftrightarrow \qquad \boldsymbol{0} = \eta_k \left[\boldsymbol{h} - \tilde{\boldsymbol{H}} \boldsymbol{x}_{k+1} \right]$$

$$\Leftrightarrow \qquad \boldsymbol{h} = \tilde{\boldsymbol{H}} \boldsymbol{x}_{k+1}$$

residual and energy function

the vector

$$\boldsymbol{r}_k = \boldsymbol{h} - \tilde{\boldsymbol{H}}\,\boldsymbol{x}_k$$

is the residual in iteration k

the function

$$E(\boldsymbol{x}) = \tfrac{1}{2}\,\boldsymbol{x}^\intercal \tilde{\boldsymbol{H}}\,\boldsymbol{x} - \boldsymbol{x}^\intercal \boldsymbol{h}$$

is called an energy function

now that we know about the energy $E(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^\intercal \tilde{\boldsymbol{H}}\boldsymbol{x} - \boldsymbol{x}^\intercal \boldsymbol{h}$, it becomes clearer why *we* write $\tilde{\boldsymbol{H}}$ to refer to matrix $\boldsymbol{M}^\intercal\boldsymbol{M}$

this is because $\tilde{\boldsymbol{H}}$ is a Hamiltonian operator (people with a background in quantum computing will know what this means ;-)

however, *our* matrix $\tilde{\boldsymbol{H}}$ must not be confused with the *Hessian matrix* $\boldsymbol{H}$ which also occurs frequently in the context of mathematical optimization ...



note

$$E(\boldsymbol{x}) = \tfrac{1}{2}\boldsymbol{x}^{\mathsf{T}}\tilde{\boldsymbol{H}}\boldsymbol{x} - \boldsymbol{x}^{\mathsf{T}}\boldsymbol{h}$$

$$\Rightarrow \quad \nabla E(\boldsymbol{x}) = \tilde{\boldsymbol{H}}\boldsymbol{x} - \boldsymbol{h}$$



note

$\Rightarrow$ the process in (1) can be written as

$$\begin{aligned} \boldsymbol{x}_{k+1} &= \boldsymbol{x}_k + \eta_k \left[\boldsymbol{h} - \tilde{\boldsymbol{H}} \boldsymbol{x}_k \right] \\ &= \boldsymbol{x}_k + \eta_k \left[-\nabla E(\boldsymbol{x}_k) \right] \\ &= \boldsymbol{x}_k - \eta_k \nabla E(\boldsymbol{x}_k) \end{aligned}$$

$\Leftrightarrow$ (1) is a *gradient descent* scheme

theorem

the *optimal* step size for (1) is

$$\eta_k = \frac{\boldsymbol{r}_k^\mathsf{T} \boldsymbol{r}_k}{\boldsymbol{r}_k^\mathsf{T} \tilde{\boldsymbol{H}} \boldsymbol{r}_k}$$

proof

defining

$$g(\eta) = E(\boldsymbol{x}_k + \eta\,\boldsymbol{r}_k) = \tfrac{1}{2}\left[\boldsymbol{x}_k + \eta\,\boldsymbol{r}_k\right]^\intercal \tilde{\boldsymbol{H}}\left[\boldsymbol{x}_k + \eta\,\boldsymbol{r}_k\right] - \left[\boldsymbol{x}_k + \eta\,\boldsymbol{r}_k\right]^\intercal \boldsymbol{h}$$

we have

$$\begin{aligned}
\frac{d}{d\eta}\, g(\eta) &= \boldsymbol{r}_k^\intercal \tilde{\boldsymbol{H}}\left[\boldsymbol{x}_k + \eta\,\boldsymbol{r}_k\right] - \boldsymbol{r}_r^\intercal \boldsymbol{h} \\
&= \boldsymbol{r}_k^\intercal \tilde{\boldsymbol{H}}\boldsymbol{x}_k + \eta\,\boldsymbol{r}_k^\intercal \tilde{\boldsymbol{H}}\,\boldsymbol{r}_k - \boldsymbol{r}_r^\intercal \boldsymbol{h} \\
&= \boldsymbol{r}_k^\intercal \left[\tilde{\boldsymbol{H}}\boldsymbol{x}_k - \boldsymbol{h}\right] + \eta\,\boldsymbol{r}_k^\intercal \tilde{\boldsymbol{H}}\,\boldsymbol{r}_k \\
&= -\boldsymbol{r}_k^\intercal \boldsymbol{r}_k + \eta\,\boldsymbol{r}_k^\intercal \tilde{\boldsymbol{H}}\,\boldsymbol{r}_k
\end{aligned}$$

$\Rightarrow$ upon equating to zero and solving for η, we obtain the claimed result $\square$

iterative LSQ

guess $\boldsymbol{x}_0$

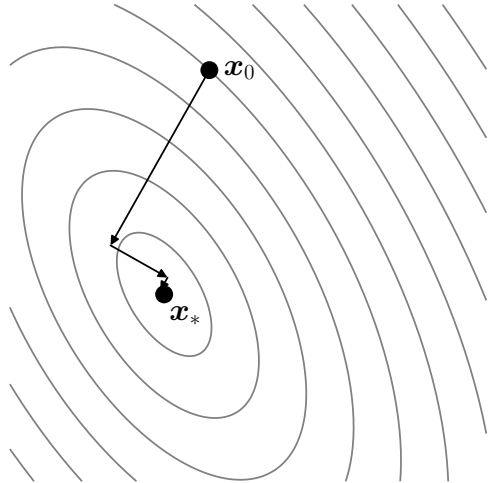
$\boldsymbol{r}_0 = \boldsymbol{h} - \tilde{\boldsymbol{H}}\,\boldsymbol{x}_0$

for $k = 0, 1, 2, \ldots$

$\boldsymbol{h} = \tilde{\boldsymbol{H}}\,\boldsymbol{r}_k$

$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \frac{\boldsymbol{r}_k^{\mathsf{T}}\boldsymbol{r}_k}{\boldsymbol{r}_k^{\mathsf{T}}\boldsymbol{h}}\,\boldsymbol{r}_k$

$\boldsymbol{r}_{k+1} = \boldsymbol{r}_k - \frac{\boldsymbol{r}_k^{\mathsf{T}}\boldsymbol{r}_k}{\boldsymbol{r}_k^{\mathsf{T}}\boldsymbol{h}}\,\boldsymbol{h}$



at first sight, the above pseudo code does not look like an implementation of (1)

however, it is an very *efficient* implementation of (1) with only one matrix-vector product per iteration . . .

convince yourself that our pseudo code really implements (1); all the equations you need have already been presented

the above example suggests that iterative least squares computation converges quickly and it generally really does

however, we can do even better . . .

since matrix $\tilde{\boldsymbol{H}}$ is symmetric and positive definite, it defines an *inner product*

$$\boldsymbol{u}^{\mathsf{T}}\tilde{\boldsymbol{H}}\,\boldsymbol{v} = \langle \boldsymbol{u}, \boldsymbol{v} \rangle_{\tilde{\boldsymbol{H}}} \equiv \langle \tilde{\boldsymbol{H}}\,\boldsymbol{u}, \boldsymbol{v} \rangle = \langle \boldsymbol{u}, \tilde{\boldsymbol{H}}^{\mathsf{T}}\boldsymbol{v} \rangle = \langle \boldsymbol{u}, \tilde{\boldsymbol{H}}\,\boldsymbol{v} \rangle$$

two vectors are *conjugate*, if they are orthogonal under such an inner product

now, let

$$C = \{\boldsymbol{c}_1, \boldsymbol{c}_2, \ldots, \boldsymbol{c}_n\}$$

be any set of conjugate vectors that span $\mathbb{R}^n$ so that we have these equations

$$\boldsymbol{x}_* = \sum_{j=1}^{n} \gamma_j \boldsymbol{c}_j \quad \text{and} \quad \tilde{\boldsymbol{H}}\,\boldsymbol{x}_* = \sum_{j=1}^{n} \gamma_j \tilde{\boldsymbol{H}}\,\boldsymbol{c}_j = \boldsymbol{h}$$

for some appropriate choice of the coefficients γ_j

to see how these coefficients should look like, we further note

$$\boldsymbol{c}_k^\intercal \boldsymbol{h} = \boldsymbol{c}_k^\intercal \tilde{\boldsymbol{H}} \boldsymbol{x}_* = \sum_{j=1}^{n} \gamma_j \boldsymbol{c}_k^\intercal \tilde{\boldsymbol{H}} \boldsymbol{c}_j = \sum_{j=1}^{n} \gamma_j \left\langle \boldsymbol{c}_k, \boldsymbol{c}_j \right\rangle_{\tilde{\boldsymbol{H}}} = \gamma_k \left\langle \boldsymbol{c}_k, \boldsymbol{c}_k \right\rangle_{\tilde{\boldsymbol{H}}} = \gamma_k \boldsymbol{c}_k^\intercal \tilde{\boldsymbol{H}} \boldsymbol{c}_k$$

this immediately yields

$$\gamma_k = \frac{\boldsymbol{c}_k^\intercal \boldsymbol{h}}{\boldsymbol{c}_k^\intercal \tilde{\boldsymbol{H}} \boldsymbol{c}_k}$$

and suggests to solve $\tilde{\boldsymbol{H}} \boldsymbol{x} = \boldsymbol{h}$ via *conjugate gradient descent*

conjugate gradients

guess $\boldsymbol{x}_0$

$\boldsymbol{r}_0 = \boldsymbol{h} - \tilde{\boldsymbol{H}}\,\boldsymbol{x}_0$
$\boldsymbol{c}_0 = \boldsymbol{h} - \tilde{\boldsymbol{H}}\,\boldsymbol{x}_0$

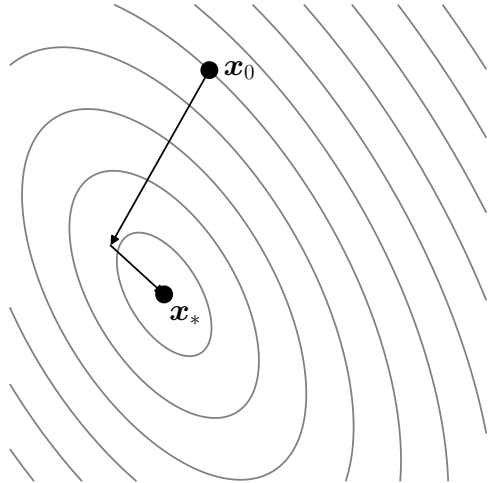
for $k = 0, 1, 2, \ldots$

$$\eta_k = \frac{\boldsymbol{r}_k^{\mathsf{T}} \boldsymbol{r}_k}{\boldsymbol{c}_k^{\mathsf{T}} \tilde{\boldsymbol{H}}\, \boldsymbol{c}_k}$$

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \eta_k\, \boldsymbol{c}_k$$

$$\boldsymbol{r}_{k+1} = \boldsymbol{r}_k - \eta_k\, \tilde{\boldsymbol{H}}\, \boldsymbol{c}_k$$

$$\boldsymbol{c}_{k+1} = \boldsymbol{r}_{k+1} - \frac{\boldsymbol{r}_{k-1}^{\mathsf{T}} \boldsymbol{r}_{k-1}}{\boldsymbol{r}_k^{\mathsf{T}} \boldsymbol{r}_k}\, \tilde{\boldsymbol{H}}\, \boldsymbol{c}_k$$



the above figure suggests that the method of conjugate gradients does converge quickly (for convex functions !!!)

indeed, as there can only be n conjugate directions in an n-dimensional space, it will converge within n steps (for convex functions !!!)

the above pseudo code looks so formidable because it involves implicit *Gram-Schmidt orthonormalization* . . . we could study this in great detail but we won't !

instead, we next (briefly) look at different flavors of gradient descent for more general settings . . .

gradient-based optimization

general problem

given a not so benign function

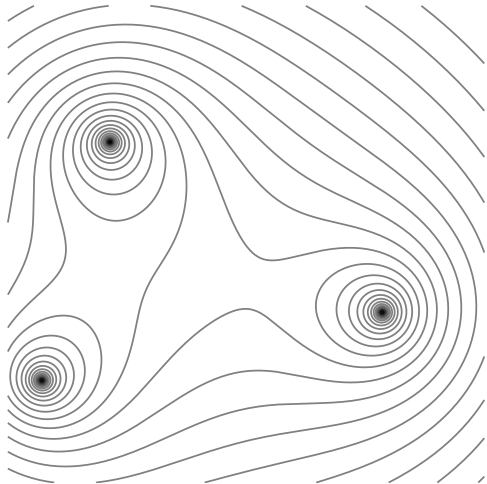
$$f : \mathbb{R}^m \to \mathbb{R}$$

for which we can compute the gradient

$$\nabla f(\boldsymbol{x}) \in \mathbb{R}^m$$

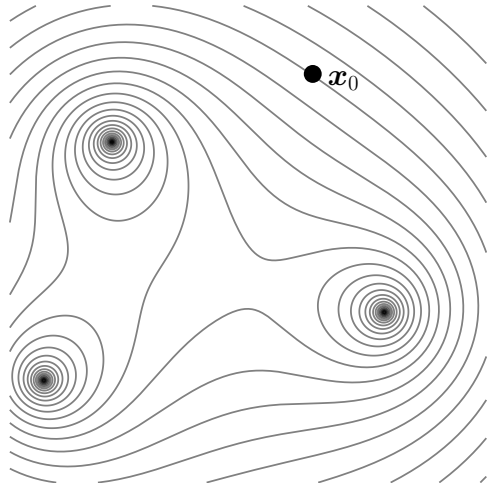
at any $\boldsymbol{x} \in \mathbb{R}^m$, we want to find

$$\boldsymbol{x}_* = \operatorname*{argmin}_{\boldsymbol{x}} f(\boldsymbol{x})$$



gradient descent (GD)

guess some possible solution $\boldsymbol{x}_0$

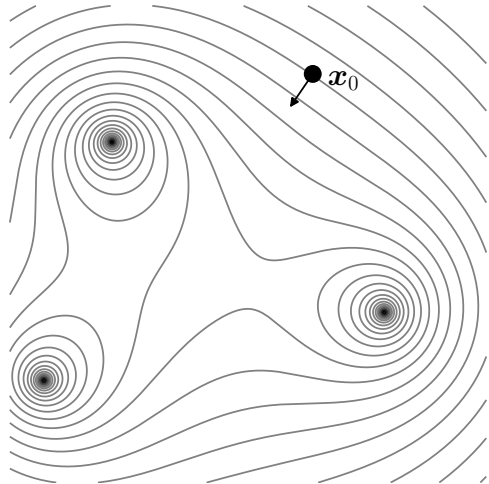


gradient descent (GD)

guess some possible solution $\boldsymbol{x}_0$

determine the negative gradient

$$\boldsymbol{g}_1 = -\nabla f(\boldsymbol{x}_0)$$



gradient descent (GD)

guess some possible solution $\boldsymbol{x}_0$

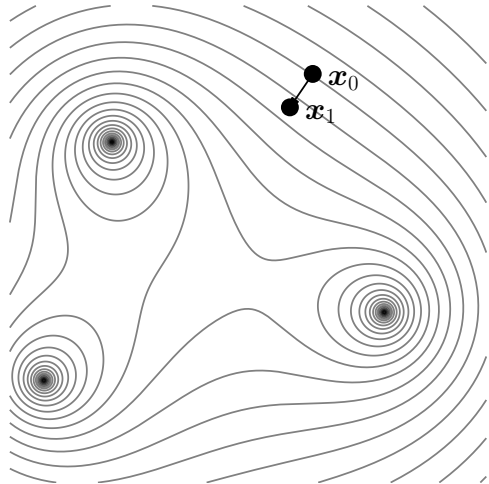
determine the negative gradient

$$\boldsymbol{g}_1 = -\nabla f(\boldsymbol{x}_0)$$

update the current estimate

$$\boldsymbol{x}_1 = \boldsymbol{x}_0 + \eta_1 \cdot \boldsymbol{g}_1$$

with step size parameter η_1



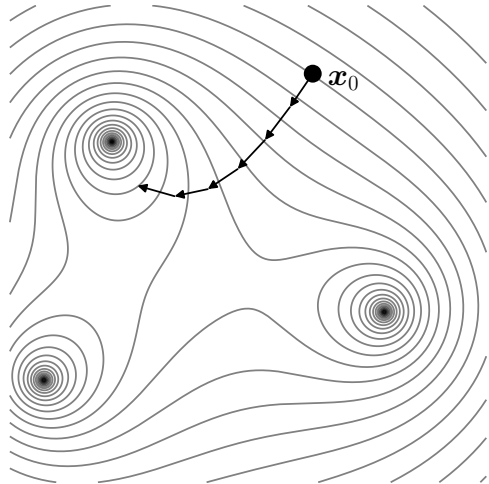
GD with fixed step size ($\forall k : \eta_k = \eta$)

guess $\boldsymbol{x}_0$

for $k = 1, 2, \ldots$

$$\boldsymbol{g}_k = -\nabla f\left(\boldsymbol{x}_{k-1}\right)$$

$$\boldsymbol{x}_k = \boldsymbol{x}_{k-1} + \eta \cdot \boldsymbol{g}_k$$



GD with fixed step size ($\forall k : \eta_k = \eta$)

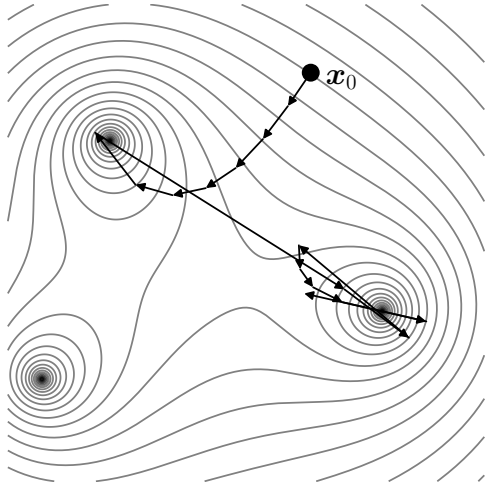
guess $\boldsymbol{x}_0$

for $k = 1, 2, \ldots$

$$\boldsymbol{g}_k = -\nabla f\left(\boldsymbol{x}_{k-1}\right)$$

$$\boldsymbol{x}_k = \boldsymbol{x}_{k-1} + \eta \cdot \boldsymbol{g}_k$$

what ???



vanishing / exploding gradients

a gradient $\nabla f(\boldsymbol{x})$ is said to

vanish, if

$$\left\|\nabla f(\boldsymbol{x})\right\| \ll 1$$

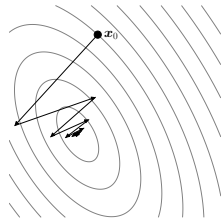
explode, if

$$\left\|\nabla f(\boldsymbol{x})\right\| \gg 1$$

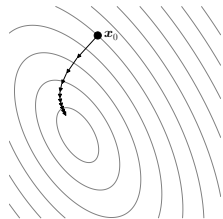
note

GD with a fixed step size is usually a bad idea, even in settings with (trivial) convex loss functions

$\Rightarrow$ humankind has put substantial efforts into devising smarter or faster or more robust GD algorithms ...



η too large $\Leftrightarrow$ overshooting



η too small $\Leftrightarrow$ slow progress

GD with line search

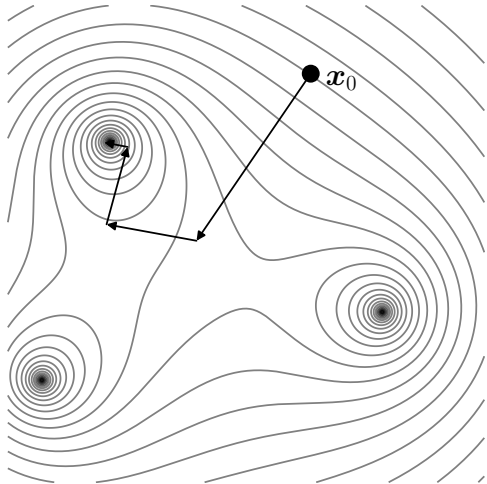
guess $\boldsymbol{x}_0$

for $k = 1, 2, \ldots$

$$\boldsymbol{g}_k = -\nabla f\left(\boldsymbol{x}_{k-1}\right)$$

$$\eta_k = \underset{\eta}{\operatorname{argmin}}\, f\left(\boldsymbol{x}_{k-1} + \eta \cdot \boldsymbol{g}_k\right)$$

$$\boldsymbol{x}_k = \boldsymbol{x}_{k-1} + \eta_k \cdot \boldsymbol{g}_k$$



in practice, we usually deal with very involved functions $f : \mathbb{R}^m \to \mathbb{R}$

$\Rightarrow$ in practice, we usually cannot exactly solve

$$\eta_k = \underset{\eta \in \mathbb{R}}{\operatorname{argmin}}\, f\left(\boldsymbol{x}_k + \eta\, \boldsymbol{g}_k\right)$$

$\Rightarrow$ in practice, we usually do discrete searches

$$\eta_k = \underset{\eta \in E}{\operatorname{argmin}}\, f\left(\boldsymbol{x}_k + \eta\, \boldsymbol{g}_k\right)$$

using, say, $E = \left\{0.01, 0.02, \ldots, 0.09, 0.1, 0.2, \ldots, 0.9, 1.0, 2.0, \ldots 9.0\right\}$

the choice of an appropriate set E is not at all trivial and very much depends on the task at hand

even if we could reliably decide for an appropriate set E, **line search is too demanding for most problems**

$\Leftrightarrow$ it becomes infeasible / too slow if we need to find optimal values of numerous (billions) of variables

but there are other ideas ...

GD with momentum (where $0 < \beta < 1$)

guess $\boldsymbol{x}_0$

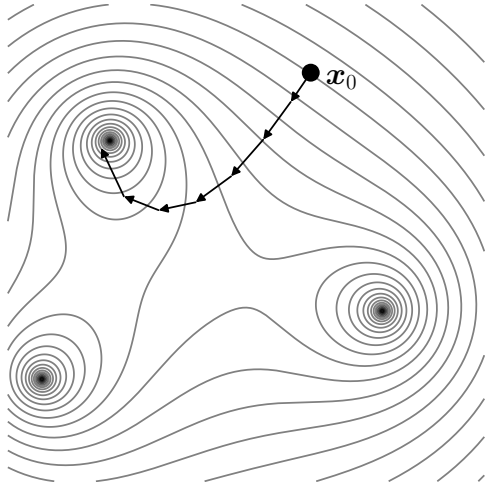
$\boldsymbol{m}_0 = \mathbf{0}$

for $k = 1, 2, \ldots$

$\boldsymbol{g}_k = -\nabla f(\boldsymbol{x}_{k-1})$

$\boldsymbol{m}_k = (1 - \beta) \cdot \boldsymbol{g}_k + \beta \cdot \boldsymbol{m}_{k-1}$

$\boldsymbol{x}_k = \boldsymbol{x}_{k-1} + \eta \cdot \boldsymbol{m}_k$



unrolling the recursion

$$\boldsymbol{m}_k = (1-\beta)\cdot \boldsymbol{g}_k + \beta \cdot \boldsymbol{m}_{k-1}$$

we obtain the following

$$\begin{aligned}
\boldsymbol{m}_1 &= (1-\beta)\cdot \boldsymbol{g}_1 + \underbrace{\beta \cdot \boldsymbol{m}_0}_{=0} \\
\boldsymbol{m}_2 &= (1-\beta)\cdot \boldsymbol{g}_2 + \beta \cdot \boldsymbol{m}_1 = (1-\beta)\cdot \boldsymbol{g}_2 + \beta \cdot (1-\beta)\cdot \boldsymbol{g}_1 \\
\boldsymbol{m}_3 &= (1-\beta)\cdot \boldsymbol{g}_3 + \beta \cdot \boldsymbol{m}_2 = (1-\beta)\cdot \boldsymbol{g}_3 + \beta \cdot (1-\beta)\cdot \boldsymbol{g}_2 + \beta^2 \cdot (1-\beta)\cdot \boldsymbol{g}_1 \\
&\vdots \\
\boldsymbol{m}_k &= (1-\beta)\sum_{j=1}^{k} \beta^{k-j}\, \boldsymbol{g}_j
\end{aligned}$$

$\Rightarrow$ the momentum technique computes a *convex combination* of the current and the previous gradient or, just as well, a *weighted sum* of all previous gradients

$\Leftrightarrow$ the momentum technique computes a kind of *average gradient* which makes it more robust against sudden changes in the *loss landscape*

this fundamental idea dates back to Polyak's pioneering work in the 1960s [1]

in the context of neural network training, it was first considered by Rumelhart, Hinton, and Williams [2] because of course it was ...

choosing an appropriate parameter β and step size η is again not at all trivial

RMSprop

guess $\boldsymbol{x}_0$

$\boldsymbol{m}_0 = \mathbf{0}$

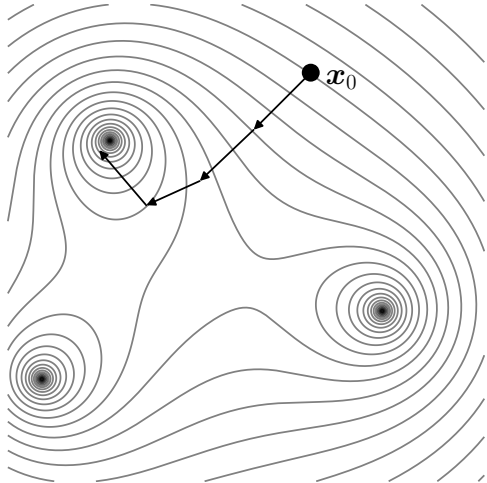
for $k = 1, 2, \ldots$

$$\boldsymbol{g}_k = -\nabla f\left(\boldsymbol{x}_{k-1}\right)$$

$$\boldsymbol{m}_k = \left(1 - \beta\right) \cdot \boldsymbol{g}_k^2 + \beta \cdot \boldsymbol{m}_{k-1}$$

$$\boldsymbol{x}_k = \boldsymbol{x}_{k-1} + \eta \cdot \frac{\boldsymbol{g}_k}{\sqrt{\boldsymbol{m}_k + \varepsilon}}$$

note: read $\boldsymbol{g}_k^2$ as $\boldsymbol{g}_k \odot \boldsymbol{g}_k$ and $\frac{\boldsymbol{g}_k}{\sqrt{\boldsymbol{m}_k + \varepsilon}}$ as $\boldsymbol{g}_k \oslash \sqrt{\boldsymbol{m}_k + \varepsilon}$



ADAM

guess $\boldsymbol{x}_0$

$\boldsymbol{m}_0 = \mathbf{0}$

$\boldsymbol{v}_0 = \mathbf{0}$

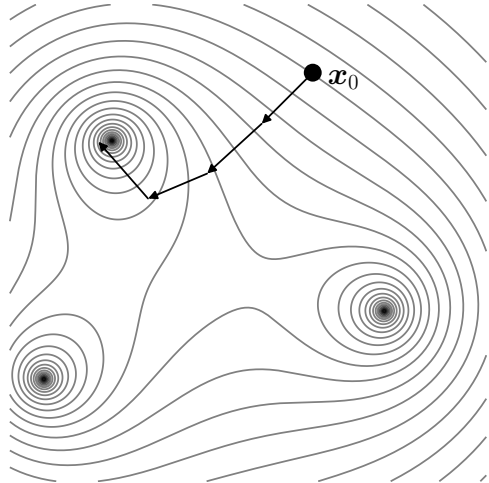
for $k = 1, 2, \ldots$

$$\boldsymbol{g}_k = -\nabla f\left(\boldsymbol{x}_{k-1}\right)$$

$$\boldsymbol{m}_k = \left(1-\beta\right) \cdot \boldsymbol{g}_k + \beta \cdot \boldsymbol{m}_{k-1}$$
$$\boldsymbol{v}_k = \left(1-\gamma\right) \cdot \boldsymbol{g}_k^2 + \gamma \cdot \boldsymbol{v}_{k-1}$$

$$\hat{\boldsymbol{m}}_k = \frac{\boldsymbol{m}_k}{\left(1-\beta^k\right)}$$
$$\hat{\boldsymbol{v}}_k = \frac{\boldsymbol{v}_k}{\left(1-\gamma^k\right)}$$

$$\boldsymbol{x}_k = \boldsymbol{x}_{k-1} + \eta \cdot \frac{\hat{\boldsymbol{m}}_k}{\sqrt{\hat{\boldsymbol{v}}_k} + \varepsilon}$$



RMSprop stems from Hinton's lab but was only disseminated as teaching material

ADAM is due to Kingma and Ba who published it at ICLR'15 [3] . . . as of today, the paper has amassed more than 260.000 citations !!!

ADAM has the following *standard parameters* (we'll say more on this (much) later)

$$\beta = 0.9$$

$$\gamma = 0.999$$

$$\eta = 0.0001$$

$$\varepsilon = 0.00000001$$

question

why is ADAM so popular ?

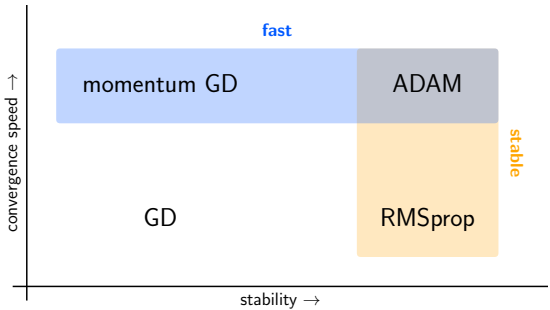
answer

because . . .



note

practitioners know and theorists can show [1, 4] the following



one more thing . . .

from exercise 2, we (already) know that we may also consider

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - \eta\, \boldsymbol{H}^{-1}\, \nabla f(\boldsymbol{x}_k)$$

where $0 < \eta \leqslant 1$ and $\boldsymbol{H}$ denotes the Hessian matrix of f at $\boldsymbol{x}_k$

$$\boldsymbol{H}\big(f(\boldsymbol{x}_k)\big) = \begin{bmatrix} \frac{\partial^2}{\partial x_1^2} f(\boldsymbol{x}_k) & \frac{\partial^2}{\partial x_1 \partial x_2} f(\boldsymbol{x}_k) & \cdots & \frac{\partial^2}{\partial x_1 \partial x_m} f(\boldsymbol{x}_k) \\ \frac{\partial^2}{\partial x_2 \partial x_1} f(\boldsymbol{x}_k) & \frac{\partial^2}{\partial x_2^2} f(\boldsymbol{x}_k) & \cdots & \frac{\partial^2}{\partial x_2 \partial x_m} f(\boldsymbol{x}_k) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2}{\partial x_m \partial x_1} f(\boldsymbol{x}_k) & \frac{\partial^2}{\partial x_m \partial x_2} f(\boldsymbol{x}_k) & \cdots & \frac{\partial^2}{\partial x_m^2} f(\boldsymbol{x}_k) \end{bmatrix}$$

$\Leftrightarrow$ Hessians gather *2nd order derivatives* of functions $f : \mathbb{R}^m \to \mathbb{R}$

Newton's method

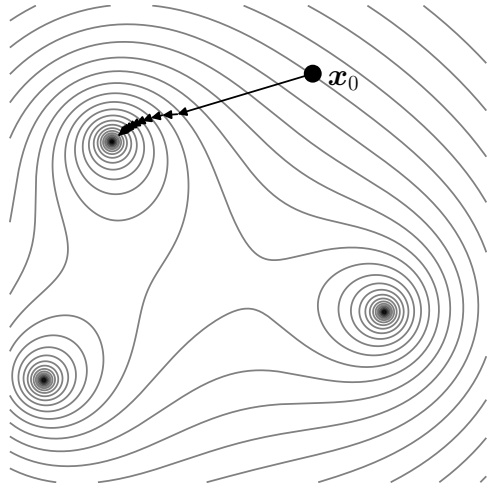
guess $\boldsymbol{x}_0$

for $k = 1, 2, \ldots$

$$\boldsymbol{g}_k = -\nabla f\big(\boldsymbol{x}_{k-1}\big)$$

$$\boldsymbol{H}_k = \boldsymbol{H}\big(f\big(\boldsymbol{x}_{k-1}\big)\big)$$

$$\boldsymbol{x}_k = \boldsymbol{x}_{k-1} + \eta\, \boldsymbol{H}_k^{-1}\, \boldsymbol{g}_k$$





note

yes, parameter η is fixed in Newton's method (when in doubt, use $\eta = 0.15$)

it is mainly the Hessian which adjusts the current step sizes and directions

as a *second order method*, Newton's method only requires few iterations to approach a (local) minimum of the function to be optimized

theoretically*, Newton's method is a near optimal optimization method, alas, *practically*, it is limited by its frequent need for *matrix inversion

$\Leftrightarrow$ Newton's method is practically infeasible if we need to find optimal values of numerous (billions) of variables

principles of machine learning

training a machine learning model ($\Leftrightarrow$ fitting a function to data) is to optimize the model's parameters (w.r.t. an objective function of the given data)

in this lecture, we are considering optimization from a general point of view and ask for minimizer(s) $\boldsymbol{x}_*$ of some function f (if we wanted to, we could also ask for maximizer(s) $\boldsymbol{x}_*$ of $-f$)

for most (practically relevant) functions f, it is impossible to determine closed form solutions for $\boldsymbol{x}_*$ so that machine learning ($\Leftrightarrow$ model training) typically requires the use of iterative optimization algorithms

nowadays, iterative optimization algorithms are almost synonymous with gradient descent techniques of which there are many

nowadays, ADAM is the most popular technique because of its favorable trade-off between efficiency and quality ($\Leftrightarrow$ ease of computation and usefulness of results)

(fast and robust methods involving inverse matrices are practically infeasible for large scale problems)

principles of machine learning

optimizers such as ADAM usually come with hyper-parameters $(\beta, \gamma, \eta, \varepsilon)$ which themselves should best be chosen optimally

alas, choosing optimal hyper-parameters is itself hardly ever feasible so that machine learners either rely on intuition, long-term experience, or tentative brute-force searches

$\Leftrightarrow$ **when we train (modern) large-scale models, we usually have no guarantee that we will end up with the best possible parametrization**

(high-dimensional optimization is a pain and we will say much more about this throughout the course)

what comes next . . .

all the approaches we studied above required computing gradients or even Hessians, i.e. first and second order *derivatives* of an objective function

however, a frequently overlooked fact in machine learning is that there exist optimization algorithms which avoid derivatives altogether

next, we therefore look at two prominent ideas for *derivative-free optimization*

these are typically on the less efficient side of computation (yet not infeasible)

their main appeal is that *they work even if derivatives are hard or impossible to obtain, for instance, when dealing with discontinuous objective functions*

gradient-free optimization

derivative-free optimization

gradient-free optimizers do not compute $\nabla f(\boldsymbol{x}_k)$ but merely evaluate the objective f at several *proposal points* $\boldsymbol{x}$ in the vicinity of the current best estimate $\boldsymbol{x}_k$

$\Leftrightarrow$ **derivative-free optimization only evaluates $f(\boldsymbol{x})$ and avoids evaluations of $\nabla f(\boldsymbol{x})$**

yet, for this to work and converge, the proposals $\boldsymbol{x}$ near $\boldsymbol{x}_k$ must be chosen cleverly ...

simultaneous perturbation stochastic approximation

SPSA due to Spall [5] roughly proceeds as follows

guess $\boldsymbol{x}_0 \in \mathbb{R}^m$

for $k = 1, 2, \ldots$

randomly draw / sample

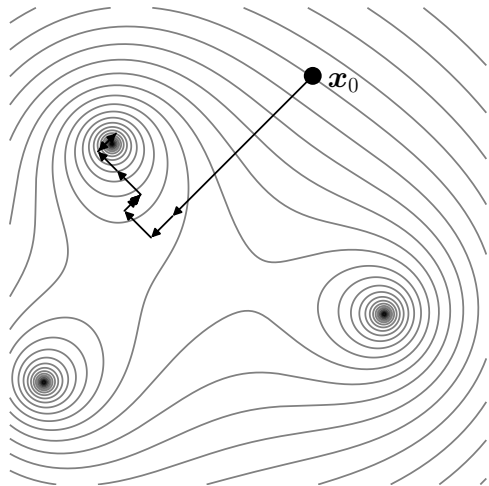
$$\boldsymbol{\delta} \sim \mathcal{U}\Big(\{-1, +1\}^m\Big)$$

$$\begin{aligned} \boldsymbol{x}^+ &= \boldsymbol{x}_{k-1} + c_k\, \boldsymbol{\delta} \\ \boldsymbol{x}^- &= \boldsymbol{x}_{k-1} - c_k\, \boldsymbol{\delta} \end{aligned}$$

$$\tilde{\boldsymbol{g}} = \frac{f(\boldsymbol{x}^+) - f(\boldsymbol{x}^-)}{2\, c_k\, \boldsymbol{\delta}}$$

$$\boldsymbol{x}_k = \boldsymbol{x}_{k-1} - a_k\, \tilde{\boldsymbol{g}}$$

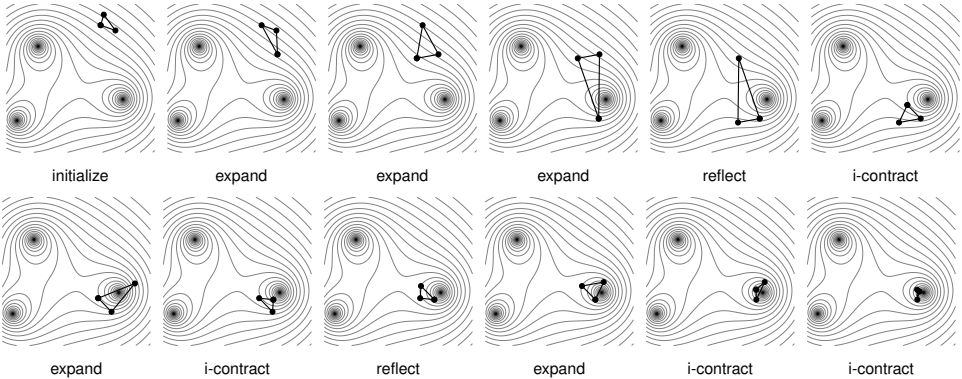
hyper-parameters a_k, c_k must be chosen carefully



the Nelder-Mead method

the Nelder-Mead [6] or simplex downhill method minimizes $f : \mathbb{R}^m \to \mathbb{R}$ by maintaining a set $X = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_{m+1}\}$ of $m + 1$ proposal points which form a *simplex* in $\mathbb{R}^m$

this simplex is iteratively updated using an intricate algorithm involving conditionally executed *geometric operations* which are called *reflection*, *expansion*, inside- and outside *contraction*, and *shrinking*



(least squares) gradient flows

recall

studying iterative least squares,
we considered the problem

$$\boldsymbol{x}_* = \operatorname*{argmin}_{\boldsymbol{x}} \left\| \boldsymbol{M}\boldsymbol{x} - \boldsymbol{v} \right\|^2$$

got to know the energy function

$$E(\boldsymbol{x}) = \tfrac{1}{2}\, \boldsymbol{x}^\intercal \boldsymbol{M}^\intercal \boldsymbol{M}\boldsymbol{x} - \boldsymbol{x}^\intercal \boldsymbol{M}^\intercal \boldsymbol{v}$$

and the iterative update scheme

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - \eta_k \nabla E(\boldsymbol{x}_k)$$

so far so good, but now ...

for what follows, we consider a fixed step size $\eta_k = \eta$ for all k so that we have

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - \eta \, \nabla E\big(\boldsymbol{x}_k\big) \tag{2}$$

$$\Leftrightarrow \qquad \frac{\boldsymbol{x}_{k+1} - \boldsymbol{x}_k}{\eta} = -\nabla E\big(\boldsymbol{x}_k\big) \tag{3}$$

as we will need it later, we also recall that the energy gradient at $\boldsymbol{x}_k$ amounts to

$$\nabla E\big(\boldsymbol{x}_k\big) = \boldsymbol{M}^\intercal \boldsymbol{M} \, \boldsymbol{x}_k - \boldsymbol{M}^\intercal \boldsymbol{v} \tag{4}$$

next, we do something crazy and *assume* a continuous function

$$\boldsymbol{x} : \mathbb{R}_+ \to \mathbb{R}^m, \quad t \mapsto \boldsymbol{x}(t)$$

which produces the above discrete iterates $\boldsymbol{x}_k$ and $\boldsymbol{x}_{k+1}$ like this

$$\begin{aligned} \boldsymbol{x}(t) &= \boldsymbol{x}_k \\ \boldsymbol{x}(t+\eta) &= \boldsymbol{x}_{k+1} = \boldsymbol{x}_k - \eta \nabla E\big(\boldsymbol{x}_k\big) = \boldsymbol{x}(t) - \eta \nabla E\big(\boldsymbol{x}(t)\big) \end{aligned}$$

assuming this function, we immediately have

$$\frac{\boldsymbol{x}(t+\eta) - \boldsymbol{x}(t)}{\eta} = -\nabla E\big(\boldsymbol{x}(t)\big) \tag{5}$$

and it become clear where we are headed ...



note

in the limit $\eta \to 0$, (5) becomes

$$\frac{d}{dt}\boldsymbol{x}(t) = -\nabla E\big(\boldsymbol{x}(t)\big) \qquad \text{(Leibnitz notation)}$$

$$\Leftrightarrow \qquad \dot{\boldsymbol{x}}(t) = -\nabla E\big(\boldsymbol{x}(t)\big) \qquad \text{(Newton notation)}$$

$\Rightarrow$ we obtain an *ordinary differential equation* or a *continuous time dynamical system* for which the gradient updates in (2) are the *forward Euler* iterates for solving it

much more crucially, we obtain an instance of a *gradient flow* where a system's state evolves in the direction of the negative gradient of a potential (energy) function

gradient flow

given a smooth differentiable function $E : \mathbb{R}^m \to \mathbb{R}$, a gradient flow is a smooth differentiable curve $\boldsymbol{x} : \mathbb{R}_+ \to \mathbb{R}^m$, $t \mapsto \boldsymbol{x}(t)$ such that

$$\dot{\boldsymbol{x}}(t) = -\nabla E\big(\boldsymbol{x}(t)\big)$$

question

but does our particular least squares flow

$$\dot{\boldsymbol{x}}(t) = -\nabla E\big(\boldsymbol{x}(t)\big) = \boldsymbol{M}^{\intercal}\boldsymbol{v} - \boldsymbol{M}^{\intercal}\boldsymbol{M}\,\boldsymbol{x}(t)$$

exist ?

in other words, does the following integral

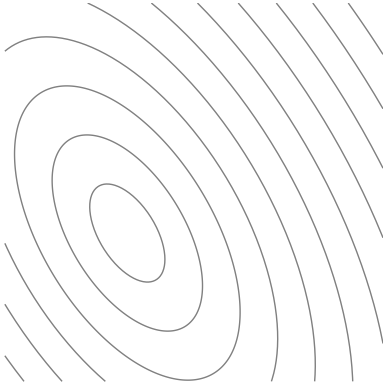
$$\boldsymbol{x}(t) = \int_0^t \Big[\boldsymbol{M}^{\intercal}\boldsymbol{v} - \boldsymbol{M}^{\intercal}\boldsymbol{M}\,\boldsymbol{x}(\tau)\Big]\,d\tau$$

exist ?

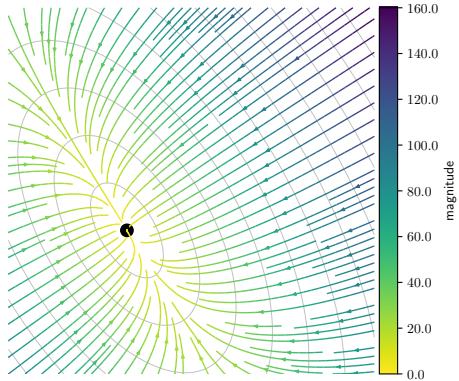
answer

yes !

example



(quadratic) energy function



gradient flow field

in fact, **as far as gradient flows go, least squares flows are rather simple**

they have a single and unique stationary point $\boldsymbol{x}_*$ where $\nabla E(\boldsymbol{x}_*) = \boldsymbol{0}$, namely

$$\boldsymbol{x}_* = \left[\boldsymbol{M}^\intercal \boldsymbol{M}\right]^{-1} \boldsymbol{M}^\intercal \boldsymbol{v}$$

this is of course our well known least squares solution for which we can verify

$$\begin{aligned}\nabla E(\boldsymbol{x}_*) &= \boldsymbol{M}^\intercal \boldsymbol{v} - \boldsymbol{M}^\intercal \boldsymbol{M}\, \boldsymbol{x}_* \\ &= \boldsymbol{M}^\intercal \boldsymbol{v} - \boldsymbol{M}^\intercal \boldsymbol{M} \left[\boldsymbol{M}^\intercal \boldsymbol{M}\right]^{-1} \boldsymbol{M}^\intercal \boldsymbol{v} = \boldsymbol{M}^\intercal \boldsymbol{v} - \boldsymbol{M}^\intercal \boldsymbol{v} = \boldsymbol{0}\end{aligned}$$

moreover, as the LSQ solution $\boldsymbol{x}_*$ is a minimizer of a quadratic energy function $E(\boldsymbol{x})$, it is also an asymptotically stable equilibrium point of the system $\dot{\boldsymbol{x}}(t) = -\nabla E\big(\boldsymbol{x}(t)\big)$

to quickly see that $\boldsymbol{x}_*$ is a stable equilibrium of the system, $\dot{\boldsymbol{x}}(t) = -\nabla E\big(\boldsymbol{x}(t)\big)$, we let

$$\begin{aligned} \tilde{\boldsymbol{H}} &= -\boldsymbol{M}^\intercal \boldsymbol{M} \\ \boldsymbol{h} &= \quad \boldsymbol{M}^\intercal \boldsymbol{v} \end{aligned}$$

and scrutinize the system

$$\dot{\boldsymbol{x}}(t) = \tilde{\boldsymbol{H}}\,\boldsymbol{x}(t) + \boldsymbol{h}$$

from dynamical systems theory / control theory we know (?!?) that an equilibrium of this system is stable, if an only if all the eigenvalues of $\tilde{\boldsymbol{H}}$ have negative real parts ...

for our $\tilde{\boldsymbol{H}}$ this is indeed the case, because

1) matrix $\boldsymbol{M}^\intercal\boldsymbol{M}$ is symmetric $\boldsymbol{M}^\intercal\boldsymbol{M} = \left[\boldsymbol{M}^\intercal\boldsymbol{M}\right]^\intercal$
2) matrix $\boldsymbol{M}^\intercal\boldsymbol{M}$ is a Gram matrix with entries

$$\left[\boldsymbol{M}^\intercal\boldsymbol{M}\right]_{kl} = \boldsymbol{m}_{k:}^\intercal\boldsymbol{m}_{:l}$$

3) matrix $\boldsymbol{M}^\intercal\boldsymbol{M}$ is therefore positive definite

$$\boldsymbol{x}^\intercal\boldsymbol{M}^\intercal\boldsymbol{M}\,\boldsymbol{x} = \sum_k\sum_l x_k\,x_l\,\boldsymbol{m}_{k:}^\intercal\boldsymbol{m}_{:l} = \left[\sum_k x_k\,\boldsymbol{m}_{:k}\right]^\intercal\left[\sum_l x_l\,\boldsymbol{m}_{:l}\right] = \left\|\sum_k x_k\,\boldsymbol{m}_{:k}\right\|^2 \geqslant 0$$

4) since $\boldsymbol{M}^\intercal\boldsymbol{M}$ is symmetric and positive definite, its eigenvalues are real and positive

$\Rightarrow$ since we have defined $\tilde{\boldsymbol{H}}$ as $\tilde{\boldsymbol{H}} = -\boldsymbol{M}^\intercal\boldsymbol{M}$, all its eigenvalues are real and negative

finally, for any *initial value* $\boldsymbol{x}(0)$, least squares flows *converge exponentially fast* to their stationary / stable equilibrium point $\boldsymbol{x}_*$

to quickly see this, we once again consider

$$\dot{\boldsymbol{x}}(t) = \tilde{\boldsymbol{H}}\,\boldsymbol{x}(t) + \boldsymbol{h}$$

and note that, at the equilibrium, we have

$$\dot{\boldsymbol{x}}_*(t) = \tilde{\boldsymbol{H}}\,\boldsymbol{x}_*(t) + \boldsymbol{h} = -\nabla E\big(\boldsymbol{x}_*(t)\big) = \boldsymbol{0}$$

$\Leftrightarrow$ once the system reaches $\boldsymbol{x}_*$, its state is stable and does not change anymore

now . . .

now, since

$$\tilde{\boldsymbol{H}}\,\boldsymbol{x}_* + \boldsymbol{h} = \boldsymbol{0} \qquad \Leftrightarrow \qquad \boldsymbol{h} = -\tilde{\boldsymbol{H}}\,\boldsymbol{x}_*$$

we can rewrite our dynamical system as

$$\dot{\boldsymbol{x}}(t) = \tilde{\boldsymbol{H}}\,\boldsymbol{x}(t) + \boldsymbol{h} = \tilde{\boldsymbol{H}}\,\boldsymbol{x}(t) - \tilde{\boldsymbol{H}}\,\boldsymbol{x}_* = \tilde{\boldsymbol{H}}\big[\boldsymbol{x}(t) - \boldsymbol{x}_*\big]$$

given an initial value $\boldsymbol{x}(0)$, the solution to this *first-order matrix differential equation* is

$$\begin{aligned}\boldsymbol{x}(t) &= \boldsymbol{x}_* + e^{t\tilde{\boldsymbol{H}}}\big[\boldsymbol{x}(0) - \boldsymbol{x}_*\big] \\ &= \big[\boldsymbol{I} - e^{t\tilde{\boldsymbol{H}}}\big]\,\boldsymbol{x}_* + e^{t\tilde{\boldsymbol{H}}}\,\boldsymbol{x}(0)\end{aligned}$$



note

matrix $\boldsymbol{I}$ denotes the identity matrix and

$$e^{t\tilde{\boldsymbol{H}}} = \sum_{k=0}^{\infty} \frac{1}{k!} \left[t\tilde{\boldsymbol{H}}\right]^k$$

is a matrix exponential

based on this definition, we can see that

$$\frac{d}{dt} e^{t\tilde{\boldsymbol{H}}} = \tilde{\boldsymbol{H}}\, e^{t\tilde{\boldsymbol{H}}}$$

$\Rightarrow$ using the above and the equivalence

$$\boldsymbol{x}(t) = \boldsymbol{x}_* + e^{t\tilde{\boldsymbol{H}}}\big[\boldsymbol{x}(0) - \boldsymbol{x}_*\big] \tag{6}$$

$$\Leftrightarrow \quad \boldsymbol{x}(t) - \boldsymbol{x}_* = e^{t\tilde{\boldsymbol{H}}}\big[\boldsymbol{x}(0) - \boldsymbol{x}_*\big] \tag{7}$$

we can verify our claim that (6) solves $\dot{\boldsymbol{x}}(t) = \tilde{\boldsymbol{H}}\big[\boldsymbol{x}(t) - \boldsymbol{x}_*\big]$, because

$$\begin{aligned}
\dot{\boldsymbol{x}}(t) &= \tfrac{d}{dt}\left[\boldsymbol{x}_* + e^{t\tilde{\boldsymbol{H}}}\big[\boldsymbol{x}(0) - \boldsymbol{x}_*\big]\right] \\
&= \tilde{\boldsymbol{H}}\, e^{t\tilde{\boldsymbol{H}}}\big[\boldsymbol{x}(0) - \boldsymbol{x}_*\big] \\
&= \tilde{\boldsymbol{H}}\,\big[\boldsymbol{x}(t) - \boldsymbol{x}_*\big] \qquad \text{(using (7))}
\end{aligned}$$

finally, since $\tilde{\boldsymbol{H}} = -\boldsymbol{M}^{\intercal}\boldsymbol{M}$ has strictly negative eigenvalues, we have

$$\lim_{t\to\infty} e^{t\tilde{\boldsymbol{H}}} = \boldsymbol{0}$$

where the decay is exponentially fast in t and which also establishes

$$\lim_{t\to\infty} \boldsymbol{x}(t) = \lim_{t\to\infty} \left[\left[\boldsymbol{I} - e^{t\tilde{\boldsymbol{H}}} \right] \boldsymbol{x}_* + e^{t\tilde{\boldsymbol{H}}} \boldsymbol{x}(0) \right] = \boldsymbol{x}_*$$

$\Leftrightarrow$ regardless of the initial value $\boldsymbol{x}(0)$, LSQ flows converge exponentially to $\boldsymbol{x}_*$

summary

we now know about

iterative least squares solvers

different gradient descent schemes for optimization

the existence of gradient-free optimization methods

the notion of gradient flows and their use in optimization

references

[1] B.T. Polyak. Some Methods of Speeding Up the Convergence of Iteration Methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5), 1964.

[2] D.E. Rumelhart, G.E. Hinton, and R.J. Williams. Learning Representations by Back-propagating Errors. *Nature*, 323(6088), 1986.

[3] D.P. Kingma and L.J. Ba. Adam: A Method for Stochastic Optimization. In *Proc. ICLR*, 2015.

[4] S. Dereich, A. Jentzen, and A. Riekert. Sharp Higher Order Convergence Rates for the ADAM Optimizer. *arXiv:2504.19426[math.OC]*, 2025.

[5] J.C. Spall. A Stochastic Approximation Technique for Generating Maximum Likelihood Parameter Estimates. In *Proc. American Control Conf.*, 1987.

[6] J.A. Nelder and R. Mead. A Simplex Method for Function Minimization. *Computer Journal*, 7(4), 1965.